

LAPORAN PRAKTIKUM
SISTEM ADMINISTRASI DAN INFORMASI TERDISTRIBUSI
PERTEMUAN 10
API PHP DENGAN DOCKER



Disusun oleh:

Nama : Hanan Fijananto
NIM : 24/538946/SV/24555
Kelas : B2
Dosen Pengampu : Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT
LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

A Tujuan

1. Mahasiswa mampu memahami konsep penggunaan Docker dan membuat PHP API menggunakan Docker
2. Mahasiswa diharapkan dapat memahami dan melakukan implementasi verifikasi API menggunakan Docker
3. Mahasiswa mampu membuat container, image, dan melakukan manajemen container menggunakan Docker

B Dasar Teori

B.1 Docker

Docker adalah layanan yang menyediakan kemampuan untuk mengemas dan menjalankan sebuah aplikasi dalam sebuah lingkungan terisolasi yang disebut dengan container. Dengan adanya isolasi dan keamanan yang memadai memungkinkan kamu untuk menjalankan banyak container di waktu yang bersamaan pada host tertentu.

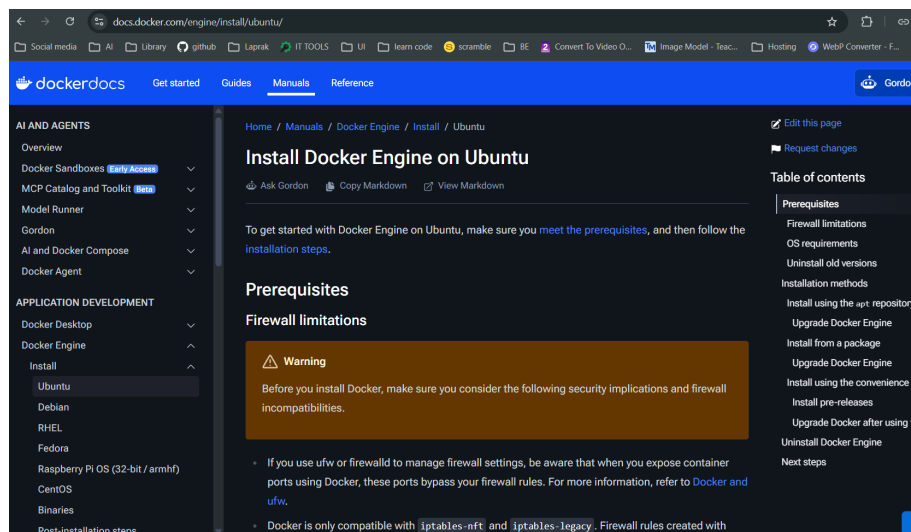
Docker ini diperkenalkan pada tahun 2013 oleh Solomon Hykes pada acara Py-Con. Beberapa bulan setelahnya docker secara resmi diluncurkan, tepatnya pada tahun 2014. Semenjak itu docker menjadi sangat populer di kalangan developer luar negeri, tetapi belum terlalu populer di Indonesia [1].

C Hasil dan Pembahasan

Pada praktikum ini, praktikan diminta untuk menghubungkan API PHP yang telah dibuat sebelumnya dengan Docker pada ubuntu server. Berikut adalah langkah-langkah yang dilakukan:

C.1 Membaca dokumentasi resmi Docker

Praktikan membaca dokumentasi resmi Docker untuk memahami konsep dasar, cara kerja, dan perintah-perintah yang digunakan dalam Docker. Dokumentasi ini sangat penting untuk memahami bagaimana Docker bekerja dan bagaimana cara menggunakannya dengan benar. Buka situs resmi Docker di <https://docs.docker.com/engine/install/ubuntu> untuk mempelajari lebih lanjut bagaimana cara menginstall Docker di VPS.



Gambar 1: Dokumentasi resmi Docker untuk instalasi di Ubuntu

C.2 Menyiapkan repository Docker di VPS

Setelah membaca dokumentasi, praktikan dapat menyiapkan repository docker di VPS dengan menjalankan perintah berikut di terminal:

1. Set up Docker's `apt` repository.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Gambar 2: Menyiapkan repository Docker di VPS

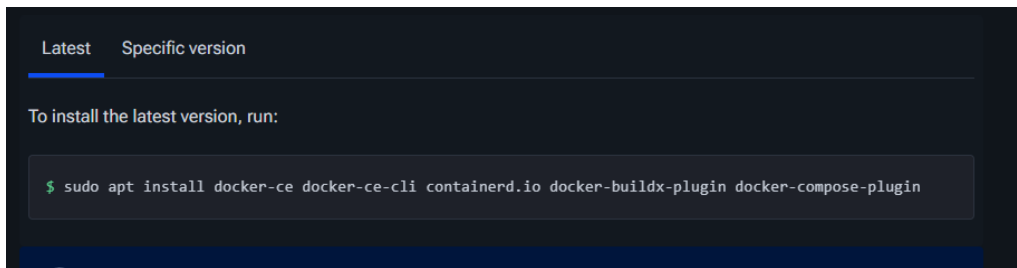
```
praktikan@psait:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
praktikan@psait:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
praktikan@psait:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
praktikan@psait:~$ sudo apt update
Get:1 http://download.zerotier.com/debian/noble noble InRelease [20.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [54.6 kB]
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:5 http://id.archive.ubuntu.com/ubuntu noble InRelease
Hit:6 http://id.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:7 http://id.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://id.archive.ubuntu.com/ubuntu noble InRelease
Fetched 124 kB in 1min 37s (1277 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
61 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Gambar 3: Hasil menyiapkan repository Docker di VPS

C.3 Menginstall Docker Engine di VPS

Selanjutnya, sesuai dokumentasi resmi Docker, praktikan dapat menginstall Docker Engine dengan menjalankan perintah berikut di terminal:

```
1 sudo apt install docker-ce docker-ce-cli containerd.io
   docker-buildx-plugin docker-compose-plugin
```

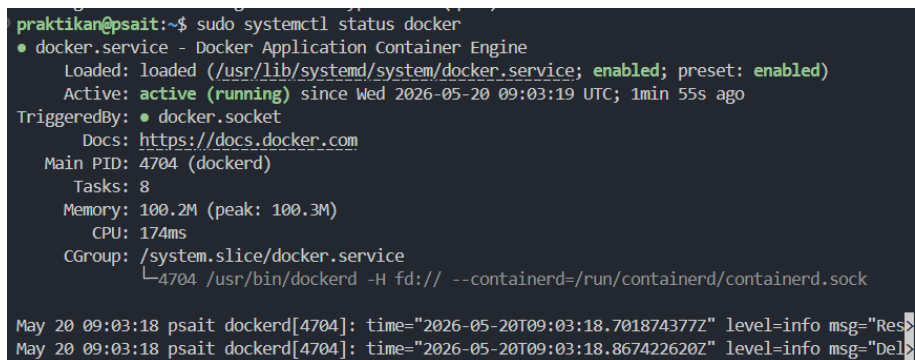


Gambar 4: Menginstall Docker Engine di VPS

C.4 Test status Docker

Praktikan dapat melakukan test apakah instalasi docker telah berhasil dengan menjalankan perintah berikut di terminal:

```
1 sudo systemctl status docker
```



Gambar 5: Test status Docker

Dapat dilihat pada gambar 5, bahwa Docker telah berhasil diinstall dan sedang berjalan dengan status active (running). Hal ini menunjukkan bahwa Docker En-

gine telah siap digunakan untuk membuat dan menjalankan container. Jika belum berjalan, dapat menjalankan perintah

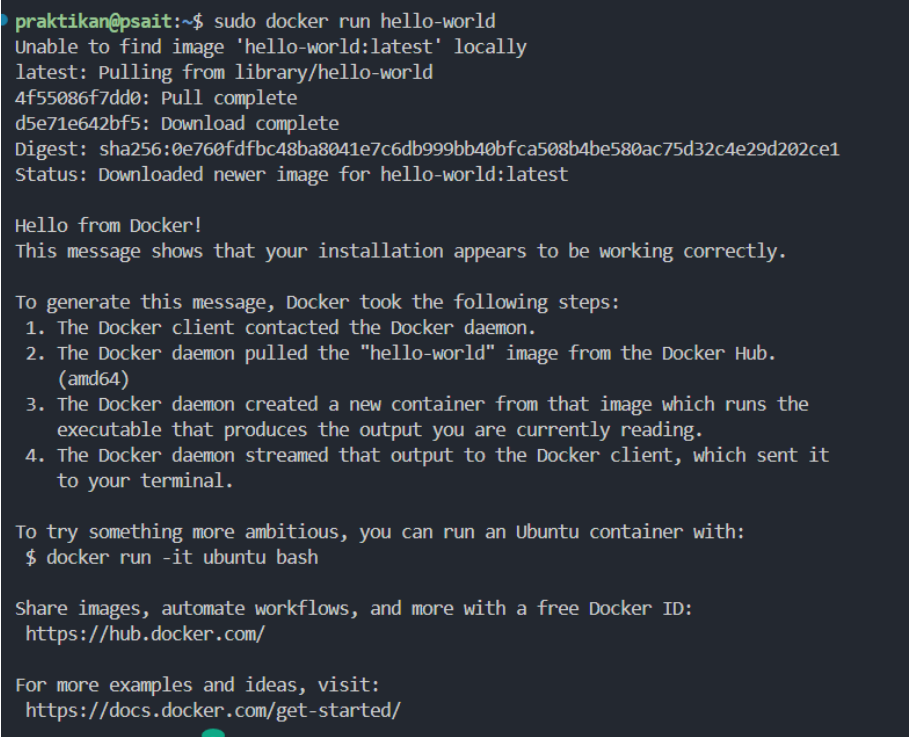
```
1 sudo systemctl start docker
```

untuk memulai layanan Docker.

C.5 Verifikasi Instalasi Docker

Praktikan dapat melakukan pengecekan sistem Docker. Docker telah menyediakan sebuah image bernama hello-world yang dapat digunakan untuk melakukan verifikasi instalasi Docker. Praktikan dapat menjalankan perintah berikut di terminal:

```
1 sudo docker run hello-world
```



```
praktikan@psait:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760dfdbc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Gambar 6: Verifikasi Instalasi Docker dengan menjalankan image hello-world

C.6 Membuat folder baru di home direktori

Praktikan dapat mencoba menggunakan Docker dengan membuat folder baru di home direktori dengan nama php-jwt-api. Praktikan dapat menjalankan seperti pada gambar 7 di terminal:

```
praktikan@psait:~$ mkdir psait
praktikan@psait:~$ cd psait
praktikan@psait:~/psait$ mkdir php-jwt-example
praktikan@psait:~/psait$ cd php-jwt-example/
praktikan@psait:~/psait/php-jwt-example$
```

Gambar 7: Membuat folder baru di home direktori dengan nama php-jwt-api

C.7 Memindahkan aplikasi ke folder baru

Setelah membuat folder baru, praktikan dapat memindahkan aplikasi PHP API (php-jwt) yang telah dibuat pada pertemuan sebelumnya ke dalam folder php-jwt-api dengan menggunakan scp (secure copy) untuk mentransfer file dari lokal ke VPS. Praktikan dapat masuk ke terminal root folder php-jwt di komputer lokal, kemudian jalankan perintah seperti pada gambar 8:

```
PS C:\laragon\www\php-jwt> scp -r . praktikan@10.33.35.71:~/psait/php-jwt-example
praktikan@10.33.35.71's password:
config                                100% 342    46.1KB/s  00:00
description                          100% 73     11.3KB/s  00:00
FETCH_HEAD                          100% 106    14.8KB/s  00:00
HEAD                                100% 21     2.9KB/s   00:00
applypatch-msg.sample               100% 478    36.1KB/s  00:00
commit-msg.sample                   100% 896    159.3KB/s 00:00
fsmonitor-watchman.sample           100% 4726   757.2KB/s 00:00
post-update.sample                  100% 189    47.0KB/s  00:00
pre-applypatch.sample               100% 424    73.9KB/s  00:00
pre-commit.sample                   100% 1649   394.3KB/s 00:00
pre-merge-commit.sample             100% 416    47.9KB/s  00:00
pre-push.sample                     100% 1374   212.2KB/s 00:00
pre-rebase.sample                   100% 4898   338.5KB/s 00:00
pre-receive.sample                  100% 544    32.6KB/s  00:00
prepare-commit-msg.sample            100% 1492   161.5KB/s 00:00
push-to-checkout.sample              100% 2783   225.0KB/s 00:00
sendemail-validate.sample            100% 2308   256.7KB/s 00:00
update.sample                       100% 3650   555.0KB/s 00:00
```

Gambar 8: Memindahkan folder php-jwt ke folder baru menggunakan scp

Setelah proses transfer selesai, praktikan dapat masuk ke folder php-jwt-api di VPS, lalu cek file database.php dan ubah sedikit konfigurasinya agar virtual net-

worknya dapat terhubung dengan localhost docker, bukan vps. Praktikan dapat mengubah konfigurasi database.php seperti pada gambar 9:

```
<?php
// used to get mysql database connection
class DatabaseService{
    private $db_host;
    private $db_name;
    private $db_user;
    private $db_password;
    private $db_port;
    private $connection;

    public function __construct(){
        $this->db_host = getenv('DB_HOST')?: 'localhost';
        $this->db_name = getenv('DB_NAME')?: 'php_jwt_db';
        $this->db_user = getenv('DB_USER')?: 'root';
        $this->db_password = getenv('DB_PASSWORD')?: '';
        $this->db_port = getenv('DB_PORT')?: '3306';
    }

    public function getConnection(){
        $this->connection = null;
        try{
            $this->connection = new PDO("mysql:host=" . $this->db_host . ";port=" . $this->db_port . ";dbname=" . $this->db_name,
            $this->db_user, $this->db_password);
        }catch(PDOException $exception){
            echo "Connection failed: " . $exception->getMessage();
        }
        return $this->connection;
    }
}
```

Gambar 9: Mengubah konfigurasi database.php untuk menghubungkan dengan localhost docker

Ubah juga konfigurasi DB_HOST di file .env agar sesuai dengan IP VPS seperti berikut:

```
1 DB_HOST = 10.33.35.71
2 DB_PORT = 3306
3 DB_USER = php_jwt
4 DB_PASSWORD = 123456
5 DB_NAME = php_jwt_db
```

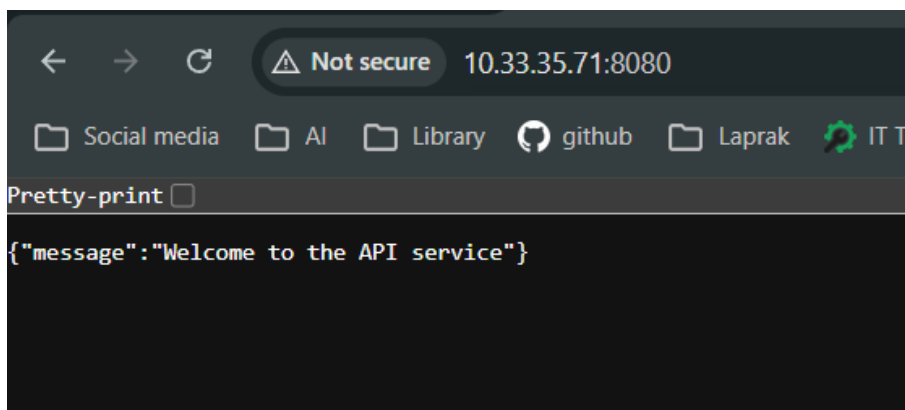
C.8 Cek aplikasi dengan development mode sebelum membuat Docker Image

Sebelum membuat Docker Image, praktikan dapat melakukan pengecekan aplikasi dengan menjalankan aplikasi dalam development mode untuk memastikan bahwa aplikasi berjalan dengan baik di lingkungan VPS. Praktikan dapat menjalankan perintah berikut di terminal:


```
praktikan@psait:~/psait/php-jwt-example$ php -S 0.0.0.0:8080
[Sat May 30 09:15:16 2026] PHP 8.3.6 Development Server (http://0.0.0.0:8080) started
```

Gambar 10: Cek aplikasi dengan development mode sebelum membuat Docker Image

Lalu, akses cek di browser dengan alamat <http://10.33.35.71:8080/>. Jika aplikasi berjalan dengan baik, maka akan muncul tampilan seperti pada gambar 11:



Gambar 11: Tampilan aplikasi berjalan dengan baik di development mode

C.9 Membuat aplikasi produksi

Praktikan menambahkan file `.htaccess` pada root projek yang bertujuan untuk membantu navigasi URL di apache beserta aturannya.

```

1  <IfModule mod_rewrite.c>
2      DirectorySlash Off
3      RewriteEngine On
4      RewriteCond %{HTTP:Authorization} ^(.*)
5      RewriteRule .* - [e=HTTP_AUTHORIZATION:%1]
6      RewriteCond %{REQUEST_FILENAME} !-f
7      RewriteRule ^ index.php [L]
8  </IfModule>
9
10

```

Gambar 12: Menambahkan file .htaccess untuk membantu navigasi URL di apache

Selanjutnya, karena folder tidak akan ditempatkan di folder di dalam html apache, maka praktikan perlu menambahkan router untuk mempermudah navigasi aplikasi.

Ubah indeks.php di root project menjadi seperti berikut

```

1  <?php
2  ob_start();
3
4  header("Access-Control-Allow-Origin: *");
5  header("Content-Type: application/json; charset=UTF-8");
6  header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
7  header("Access-Control-Max-Age: 3600");
8  header("Access-Control-Allow-Headers:
9      Content-Type, Access-Control-Allow-Headers,
10     Authorization, X-Requested-With");
11
12 $requestUri = $_SERVER['REQUEST_URI'];
13 $requestUri = strtok($requestUri, '?');
14 if ($requestUri !== '/' && substr($requestUri, -1) ===
15     '/') {
16     $requestUri = rtrim($requestUri, '/');
17 }

```

```

16 switch ($requestUri) {
17
18     case '/api/login':
19         if (file_exists('api/login/index.php')) {
20             require 'api/login/index.php';
21         } else {
22             http_response_code(500);
23             echo json_encode(["message" => "File
24                 api/login/index.php tidak ditemukan!"]);
25         }
26         break;
27
28     case '/api/register':
29         if (file_exists('api/register/index.php')) {
30             require 'api/register/index.php';
31         } else {
32             http_response_code(500);
33             echo json_encode(["message" => "File
34                 api/register/index.php tidak ditemukan!"]);
35         }
36         break;
37
38     case '/api/users':
39         if (file_exists('api/users/index.php')) {
40             require 'api/users/index.php';
41         } else {
42             http_response_code(500);
43             echo json_encode(["message" => "File
44                 api/users/index.php tidak ditemukan!"]);
45         }
46     }

```

```

43         break;
44
45     case '/':
46     case '/index.php':
47         http_response_code(200);
48         echo json_encode(array("message" => "Welcome to
49             the API service"));
50         break;
51
52     default:
53         http_response_code(404);
54         echo json_encode(array(
55             "message" => "Endpoint not found.",
56             "debug_uri" => $requestUri
57         ));
58         break;
59     }
60 }
61 ?>

```

Selanjutnya, ubah import-import di file api/login/index.php, api/register/index.php, dan api/users/index.php menjadi seperti berikut:

```

api/login/index.php
<?php
include_once __DIR__ . '/../../config/database.php';
require __DIR__ . '/../../vendor/autoload.php';
use \Firebase\JWT\JWT;

```

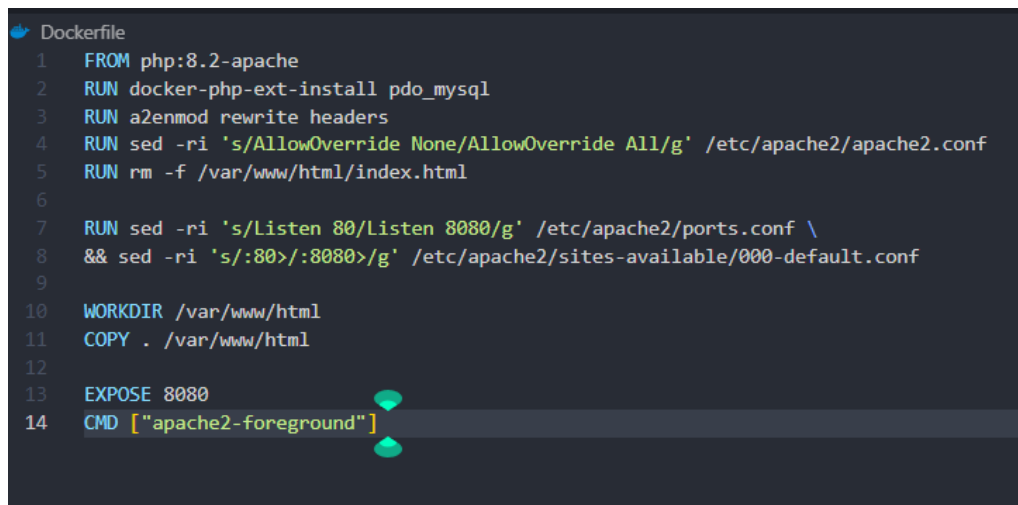
Gambar 13: Mengubah import di file api/login/index.php, api/register/index.php, dan api/users/index.php

Konstanta `__DIR__` yang praktikan tambahkan digunakan untuk memastikan bahwa impor tersebut dibaca dari lokasi file berada, jika tidak dilakukan seperti

itu, lokasi impor dapat berbeda dan dapat menyebabkan navigasi impor yang salah.

C.10 Membuat Script Dockerfile

Script Dockerfile digunakan untuk membuat Docker Image yang berisi aplikasi PHP API yang telah dibuat. Praktikan dapat membuat file Dockerfile di root project dengan isi seperti berikut:

A screenshot of a code editor showing a Dockerfile script. The script is titled 'Dockerfile' and contains 14 lines of code. The code is as follows:

```
1 FROM php:8.2-apache
2 RUN docker-php-ext-install pdo_mysql
3 RUN a2enmod rewrite headers
4 RUN sed -ri 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf
5 RUN rm -f /var/www/html/index.html
6
7 RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf \
8 && sed -ri 's/:80>/:8080>/g' /etc/apache2/sites-available/000-default.conf
9
10 WORKDIR /var/www/html
11 COPY . /var/www/html
12
13 EXPOSE 8080
14 CMD ["apache2-foreground"]
```

Gambar 14: Script Dockerfile untuk membuat Docker Image

Berdasarkan gambar 14, Secara garis besar, proses pembangunan image dimulai dengan menentukan fondasi sistem melalui perintah `FROM php:8.2-apache`, yang menetapkan bahwa kontainer akan menggunakan sistem operasi dasar dengan PHP versi 8.2 dan peladen web (web server) Apache yang telah terpasang. Selanjutnya, perintah `RUN docker-php-ext-install pdo_mysql` dieksekusi untuk memasang ekstensi `pdo_mysql` ke dalam lingkungan PHP. Ekstensi ini sangat krusial karena bertugas sebagai jembatan pembuka akses agar aplikasi PHP mampu terhubung dan berkomunikasi dengan sistem basis data MySQL. Kemudian, instruksi `RUN a2enmod rewrite headers` dijalankan untuk mengaktifkan dua modul penting pada Apache. Modul `rewrite` (`mod_rewrite`) diperlukan agar server mengenali fitur manipulasi arah URL (seperti routing API menggunakan berkas `.htaccess`), sedangkan

modul headers diperlukan untuk mengatur tajuk (header) pada respons HTTP asinkron, seperti pengaturan izin CORS (Cross-Origin Resource Sharing).

Agar peladen web mengizinkan berkas .htaccess berjalan semestinya, terdapat perintah pemrosesan teks tingkat lanjut (sed) untuk menyunting nilai AllowOverride None menjadi AllowOverride All di dalam konfigurasi induk /etc/apache2/apache2.conf. Langkah berikutnya diikuti dengan perintah `rm -f /var/www/html/index.html` yang berfungsi untuk menghapus halaman bawaan (index.html) dari instalasi awal Apache. Hal ini ditujukan supaya berkas bawaan tersebut tidak bertabrakan penamaannya dengan halaman indeks utama milik aplikasi PHP (seperti index.php) yang hendak kita jalankan. Setelah itu, modifikasi porta (port) jaringan dilakukan secara otomatis juga dengan perintah sed; instruksi tersebut mendikte peladen Apache agar mengubah porta yang mulanya berjalan standar pada porta 80 menjadi porta target 8080. Perubahan nomor porta ini dilakukan pada berkas ports.conf serta berkas peladen virtual 000-default.conf, yang berguna agar tidak terjadi tumpang tindih (conflict) apabila porta 80 pada ruang sistem (host) telah digunakan oleh layanan lain.

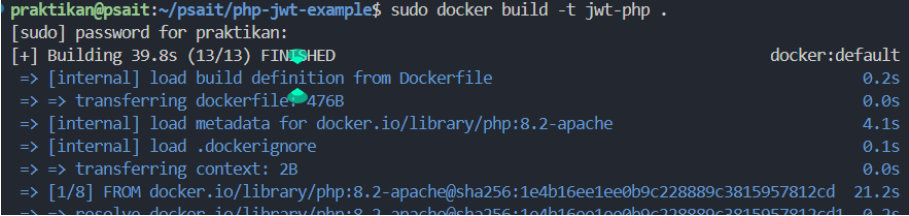
Pada tahap pemindahan kode sumber aplikasi, perintah `WORKDIR /var/www/html` menetapkan direktori tersebut sebagai direktori kerja utama (working directory). Dengan penetapan ini, instruksi selanjutnya yaitu `COPY . /var/www/html` akan menyalin instan seluruh berkas-berkas aplikasi PHP yang ada di mesin lokal saat membangun Docker (build context) ke dalam direktori pengelolaan web server tersebut di dalam container. Tahapan ini diakhiri dengan instruksi `EXPOSE 8080`, yang secara teknis berfungsi sebagai dokumentasi jaringan untuk memberitahu Docker bahwa container ini akan mengizinkan dan menerima lalu lintas permintaan jaringan pada porta 8080. Akhirnya, baris penutup `CMD ["apache2-foreground"]` adalah pengaturan perintah bawaan yang memaksa layanan Apache untuk terus berjalan pada mode tampilan utama layar (foreground). Ini merupakan langkah pamungkas yang wajib ada, karena dengan menjalankan Apache pada mode tersebut, siklus hidup

sistem operasi di dalam container akan dibiarkan terus berjalan tanpa henti untuk melayani aplikasi perantara dan tidak akan terhenti mendadak.

C.11 Build Docker Image

Setelah membuat Dockerfile, praktikan dapat melakukan build Docker Image dengan menjalankan perintah berikut di terminal:

```
1 docker build -t php-jwt-example .
```

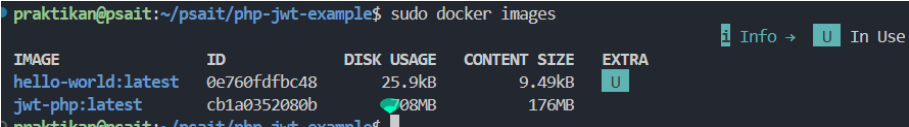


```
praktikan@psait:~/psait/php-jwt-example$ sudo docker build -t jwt-php .
[sudo] password for praktikan:
[+] Building 39.8s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 476B                               0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache   4.1s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                       0.0s
=> [1/8] FROM docker.io/library/php:8.2-apache@sha256:1e4b16ee1ee0b9c228889c3815957812cd 21.2s
=> => resolve docker.io/library/php:8.2-apache@sha256:1e4b16ee1ee0b9c228889c3815957812cd 0.3s
```

Gambar 15: Build Docker Image dengan perintah docker build

Setelah proses build selesai, praktikan dapat melihat daftar Docker Image yang telah dibuat dengan menjalankan perintah berikut di terminal:

```
1 sudo docker images
```



```
praktikan@psait:~/psait/php-jwt-example$ sudo docker images
INFO -> U In Use
IMAGE          ID              DISK USAGE  CONTENT SIZE  EXTRA
hello-world:latest 0e760fd9bc48    25.9kB      9.49kB        U
jwt-php:latest    cb1a0352080b    108MB       176MB
```

Gambar 16: Melihat daftar Docker Image yang telah dibuat

C.12 Menjalankan Image sebagai Container

Setelah membuat images, aplikasi belum dapat dijalankan secara langsung tanpa menggunakan php development server. Oleh karena itu, praktikan dapat menjalankan image yang telah dibuat sebagai container dengan menjalankan perintah berikut di terminal:

```
1 docker run -d -p 8081:8080 --env-file .env --name
    jwt-container php-jwt-example
```

```
jwt-container
praktikan@psait:~/psait/php-jwt-example$ sudo docker run -d -p 8081:8080 --env-file .env --name jwt-container jwt-php
6bba6d060ff557dfc3b8e9bc2d6e22b1c85bd10f71668a0fef55092e785b75d2
```

Gambar 17: Menjalankan Image sebagai Container dengan perintah docker run

Melalui perintah tersebut, praktikan menjalankan sebuah kontainer Docker baru berbasis image yang ditentukan di latar belakang (detached mode), sehingga terminal tetap dapat digunakan untuk instruksi selanjutnya. Dalam proses ini, praktikan memberikan identitas khusus pada kontainer dengan nama `jwt-container` untuk mempermudah manajemen dan pelacakan. Selain itu, praktikan melakukan pemetaan port 8080 pada perangkat lokal ke port 8080 di dalam kontainer, yang memungkinkan aplikasi di dalamnya dapat diakses dari luar melalui peramban web. Terakhir, praktikan mengintegrasikan file `.env` ke dalam kontainer untuk menyuntikkan seluruh variabel lingkungan (environment variables) yang diperlukan, sehingga konfigurasi aplikasi dapat termuat secara otomatis dan aman saat kontainer mulai beroperasi.

Setelah menjalankan container, praktikan dapat melihat daftar container yang sedang berjalan dengan menjalankan perintah berikut di terminal:

```
1 sudo docker ps -a
```

```
11713 pts/4    00:00:00 ps
praktikan@psait:~/psait/php-jwt-example$ sudo docker ps -a
```

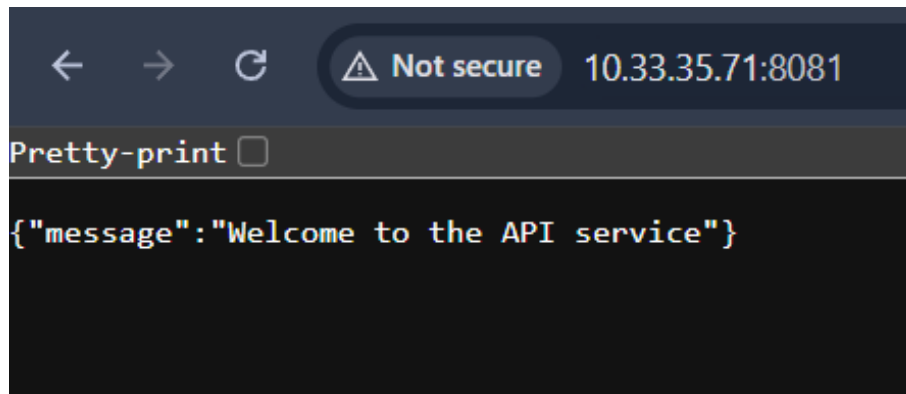
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
6bba6d060ff5	jwt-php	"docker-php-entrypoi..."	47 seconds ago	Up 45 seconds	80/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp	jwt-container
62ce8b190736	hello-world	"/hello"	59 minutes ago	Exited (0) 59 minutes ago		optimistic_aryabhata
082c2a5d998b	hello-world	"/hello"	About an hour ago	Exited (0) About an hour ago		nifty_brahmagupta

```
praktikan@psait:~/psait/php-jwt-example$
```

Gambar 18: Melihat daftar Container yang sedang berjalan

C.13 Cek aplikasi di Browser

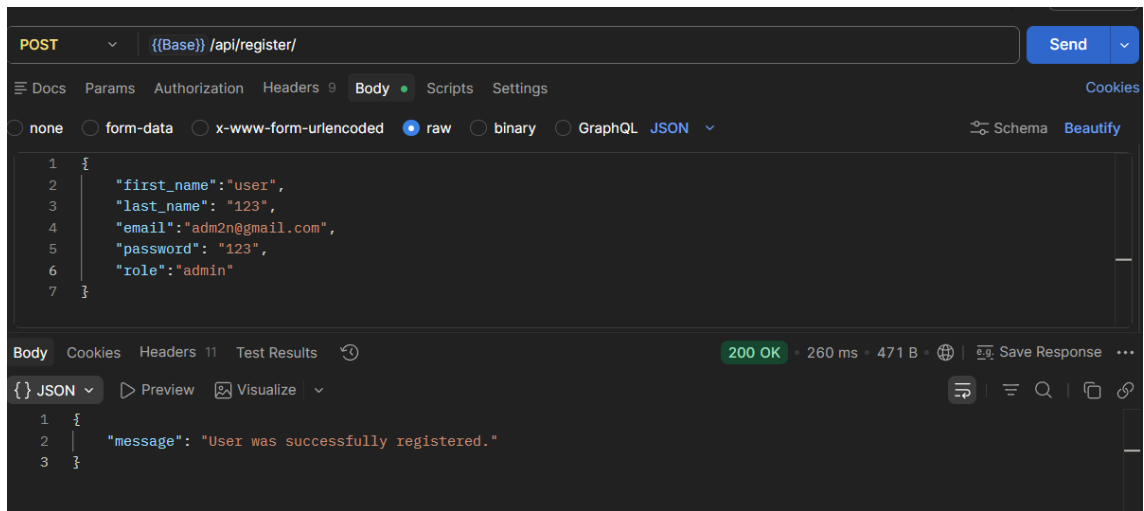
Setelah menjalankan container, dan melihat statusnya sudah up, praktikan dapat mengakses aplikasi di browser dengan alamat `http://10.33.35.71:8081`. Jika aplikasi berjalan dengan baik, maka akan muncul tampilan seperti pada gambar 19:



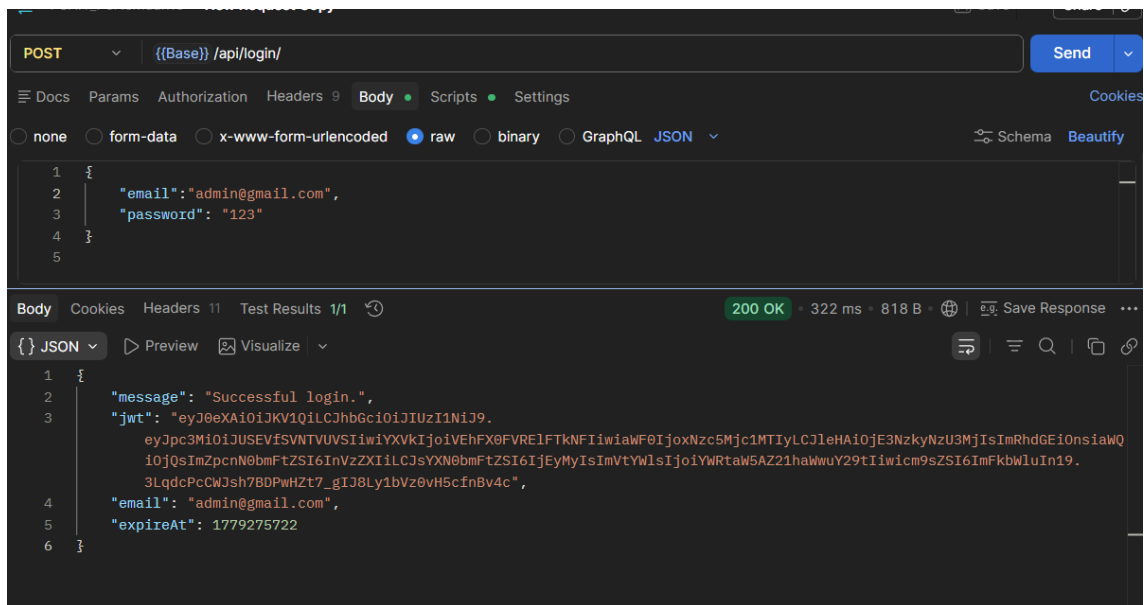
Gambar 19: Tampilan aplikasi berjalan dengan baik di dalam Container

C.14 Uji di Postman

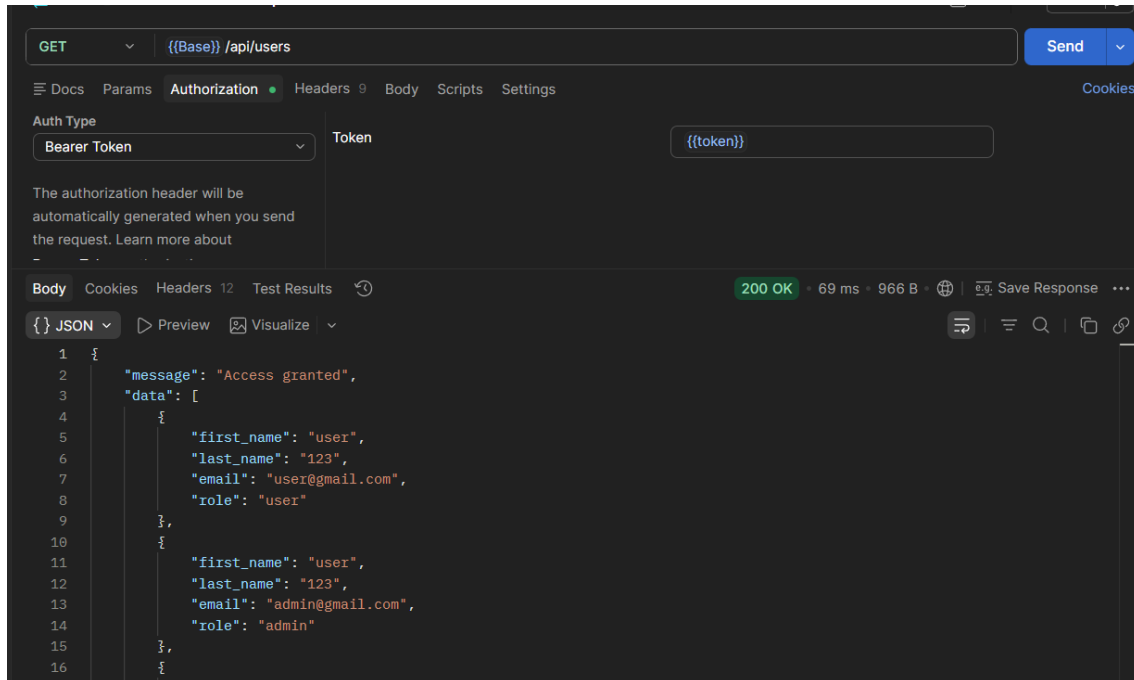
Praktikan dapat menguji aplikasi yang berjalan di dalam container dengan menggunakan Postman untuk melakukan request ke endpoint API. Praktikan dapat melakukan request ke endpoint `register`, `login`, dan `users` untuk memastikan bahwa API berfungsi dengan baik.



Gambar 20: Uji API register dengan Postman



Gambar 21: Uji API login dengan Postman



Gambar 22: Uji API users dengan Postman

D Tugas dan Analisis

1. Bagaimana cara menghapus Image Container Docker php-jwt yang telah dibuat?

Jawab:

Untuk menghapus Image Container Docker php-jwt yang telah dibuat, containernya harus dihapus terlebih dahulu. Praktikan dapat melihat terlebih dahulu daftar container yang sedang berjalan dengan perintah:

```
1 docker ps -a
```

Setelah melihat daftar container, praktikan dapat menghentikan container (jwt-container) yang sedang berjalan dengan perintah:

```
1 docker stop jwt-container
```

Setelah container dihentikan, praktikan dapat menghapus container tersebut dengan perintah:

```
1 docker rm jwt-container
```

```
praktikan@psait:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS                               NAMES
46d5d635f469   phpmyadmin:latest "/docker-entrypoint..." 4 days ago Up 4 days    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   phpmyadmin_jwt
53a310edf586   php-jwt-example-be-jwt "docker-php-entrypoint..." 4 days ago Up 4 days    80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   jwt-container
f7773a835bc4   mysql:8.0     "docker-entrypoint.s..." 4 days ago Up 4 days    3306/tcp, 33060/tcp                    db_mysql_jwt
62ce8b198736   hello-world   "/hello"                 5 days ago Exited (0) 5 days ago                    optimistic_aryabhata
082c2a5d998b   hello-world   "/hello"                 5 days ago Exited (0) 5 days ago                    nifty_brahmagupta

praktikan@psait:~/psait/php-jwt-example$ docker stop jwt-container
jwt-container
praktikan@psait:~/psait/php-jwt-example$ docker rm jwt-container
jwt-container
praktikan@psait:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS                               NAMES
46d5d635f469   phpmyadmin:latest "/docker-entrypoint..." 4 days ago Up 4 days    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   phpmyadmin_jwt
f7773a835bc4   mysql:8.0     "docker-entrypoint.s..." 4 days ago Up 4 days    3306/tcp, 33060/tcp                    db_mysql_jwt
62ce8b198736   hello-world   "/hello"                 5 days ago Exited (0) 5 days ago                    optimistic_aryabhata
082c2a5d998b   hello-world   "/hello"                 5 days ago Exited (0) 5 days ago                    nifty_brahmagupta
praktikan@psait:~/psait/php-jwt-example$
```

Gambar 23: Menghapus Container Docker jwt-container

Setelah container dihapus, praktikan dapat menghapus image php-jwt-example dengan perintah:

```
1 docker rmi php-jwt-example
```

2. Bagaimana jika source code terdapat perubahan dan Image dan Container ingin diperbarui?

Jawab:

Jika source code terdapat perubahan dan Image serta Container ingin diperbarui, praktikan dapat melakukan build ulang (rebuild) karena Docker image bersifat immutable. Praktikan dapat melakukan build ulang dengan menghentikan container yang sedang berjalan, lalu hapus container tersebut, kemudian buat ulang Docker Image dengan perintah berikut:

```
1 docker build -t php-jwt-example.
```

Pastikan nama image yang digunakan sama dengan nama image sebelumnya agar tidak terjadi duplikasi image. Setelah build ulang selesai, praktikan dapat menjalankan container baru dengan perintah:

```
1 docker run -d -p 8081:8080 --env-file .env --name
    jwt-container php-jwt-example
```

E Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Docker adalah sebuah platform yang memungkinkan untuk mengemas aplikasi beserta semua dependensinya ke dalam sebuah container yang dapat dijalankan di berbagai lingkungan. Dengan menggunakan Docker, praktikan dapat dengan mudah membuat, mengelola, dan menjalankan aplikasi tanpa harus khawatir tentang perbedaan lingkungan atau konfigurasi. Praktikum ini memberikan pemahaman tentang cara menghubungkan API PHP dengan Docker, serta bagaimana melakukan verifikasi dan manajemen container menggunakan Docker. Dengan demikian, Docker dapat menjadi alat yang sangat berguna untuk pengembangan dan deployment aplikasi secara efisien dan konsisten.

Untuk tugas tambahan, praktikan dapat menghapus Image Container Docker `php-jwt` yang telah dibuat dengan menghentikan container terlebih dahulu, lalu menghapus container tersebut, dan akhirnya menghapus image `php-jwt-example`. Jika terdapat perubahan pada source code dan ingin memperbarui Image serta Container, praktikan dapat melakukan build ulang (rebuild) dengan menghentikan container yang sedang berjalan, menghapus container tersebut, membuat ulang Docker Image dengan perintah `build`, dan menjalankan container baru dengan perintah `run`.

F Daftar Pustaka

- [1] R. Setiawan, “Apa Itu Docker? Apa Kegunaan dan Kelebihannya? - Dicoding Blog — dicoding.com,” <https://www.dicoding.com/blog/apa-itu-docker/>, 2021, [Accessed 21-05-2026].